GİRİŞ

Eğitmen

Elif ÖZKAN



linkedin.com/in/eliffozkan/
github.com/eliffozkan

Özgür YILDIRIM



linkedin.com/in/Ozgur-yldrm/
github.com/OzgurYldrm

Sunum + Notebook + Homework → Github

- Ders 1: Accuracy, Precision, Recall, F1
- ROC-AUC ve train/validation/test
- Ders 2: K-Fold ve genelleme
- Grid / Random Search
- Belirsizlik ve Bayesian: kısa tanıtım

Accuracy

Gerçek Değer	Tahmin
1	1
1	0
0	1

Accuracy = ⅓ = %33

$$\text{Accuracy} = \% \frac{\text{Doğru Sayısı}}{\text{Tüm Örnek Sayı}} \text{ x } 100$$

- **En temel metrictir.**
- **Her zaman mantıksal bir sonuç vermeyebilir.**

Pozitif sayısı: 1
Negatif Sayısı: 99
Toplam veri: 100
Böyle bir durumda sadece negatif tahmin ederek %99 doğruluk elde ettik.

Daha bilgilendirici metric lazım

Confusion Matrix

Tahmin Edilen

Gerçek Değerler

	Spam	Non-spam
Spam	True Positive	False Negative
Non-spam	False Positive	True Negative

Spam: 1

Non-spam: 0

*Contingency Table

Birçok metrik için temel oluşturan bir tablodur.

Precision

	Spam	Non-spam
Spam	8	2
Non-spam	1	9

Precision = 8/9 ≈ 0.89

	Spam	Non-spam
Spam	1	4
Non-spam	2	93

Precision = 1/3 ≈ 0.33

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

Tahmin edilen olumlu (pozitif) örneklerin ne kadarı gerçekten olumlu?

Recall

	Spam	Non-spam
Spam	8	2
Non-spam	1	9

Recall = 8/10 = 0.80

	Spam	Non-spam
Spam	1	4
Non-spam	2	93

Recall = 1/5 = 0.20

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

Gerçek olumlu (pozitif) örneklerin ne kadarını doğru tahmin ettik?

Precision-Recall Tradeoff

p ≥ eşik → pozitif; p < eşik → negatif.
Eşik azalırsa daha çok örnek pozitif olur; recall azalmaz.
Eşik artarsa daha az örnek pozitif olur; precision artabilir ama garanti değildir.

Kanser örneğinde pozitif = hasta: hasta kişiyi kaçırmamak için recall önemlidir.
Spam örneğinde pozitif = spam: doğru maili spam saymamak için precision önemlidir.
Notebookun kanser verisinde ise 1 = iyi huylu; pozitif sınıfı daima belirt.

Imbalanced Dataset → Sınıfların dağılış oranının dengesiz olduğu veri setidir. %99 negatif, %1 pozitif sınıf vb.

F1 score

Precision	Recall	F1
0.95	0.1	0.18
0.1	0.95	0.18
0.5	0.5	0.5
0.95	0.85	0.89

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Precision ve Recall arasında dengeli bir metrik

F1 harmonik ortalama mantığına dayanır. Aritmetik ortalamadan farkı değerler arasındaki büyük uçurumları daha fazla cezalandırır.

F1 score

Ground Truth	Prediction
0	0
0	1
1	1
1	0
2	2
2	2

Bir önceki slayt binary f1 score. Peki birden fazla sınıf varsa nasıl hesaplayacağız?

- **Her bir class için diğer classlar negatif kabul edilip hesaplanır**

F1 score

Ground Truth	Prediction
0	0
0	1
1	1
1	0
2	2
2	2

- **Macro F1** Her bir sınıf için F1 hesapla ortalama al

Class 0 F1 Score = 0.5

Class 1 F1 Score = 0.5 Macro F1 = ⅔ = 0.66

Class 2 F1 Score = 1.0

- **Micro F1** Her class için tp-fp-fn hesapla, topla, f1 hesapla

Class 0	Class 1	Class 2
tp: 1	tp: 1	tp: 2
fp: 1	fp: 1	fp: 0
fn: 1	fn: 1	fn: 0

TP = 4

FP = 2

FN = 2

Precision = 0.66

Recall = 0.66

Micro F1 = 0.66

F1 score

Ground Truth	Prediction
0	0
0	1
1	1
1	0
2	2
2	2

- **Weighted F1**

Class 0: 2/6

Class 1: 2/6

Class 2: 2/6

Her sınıfın F1'ını gerçek örnek sayısına göre ağırlıklandır.

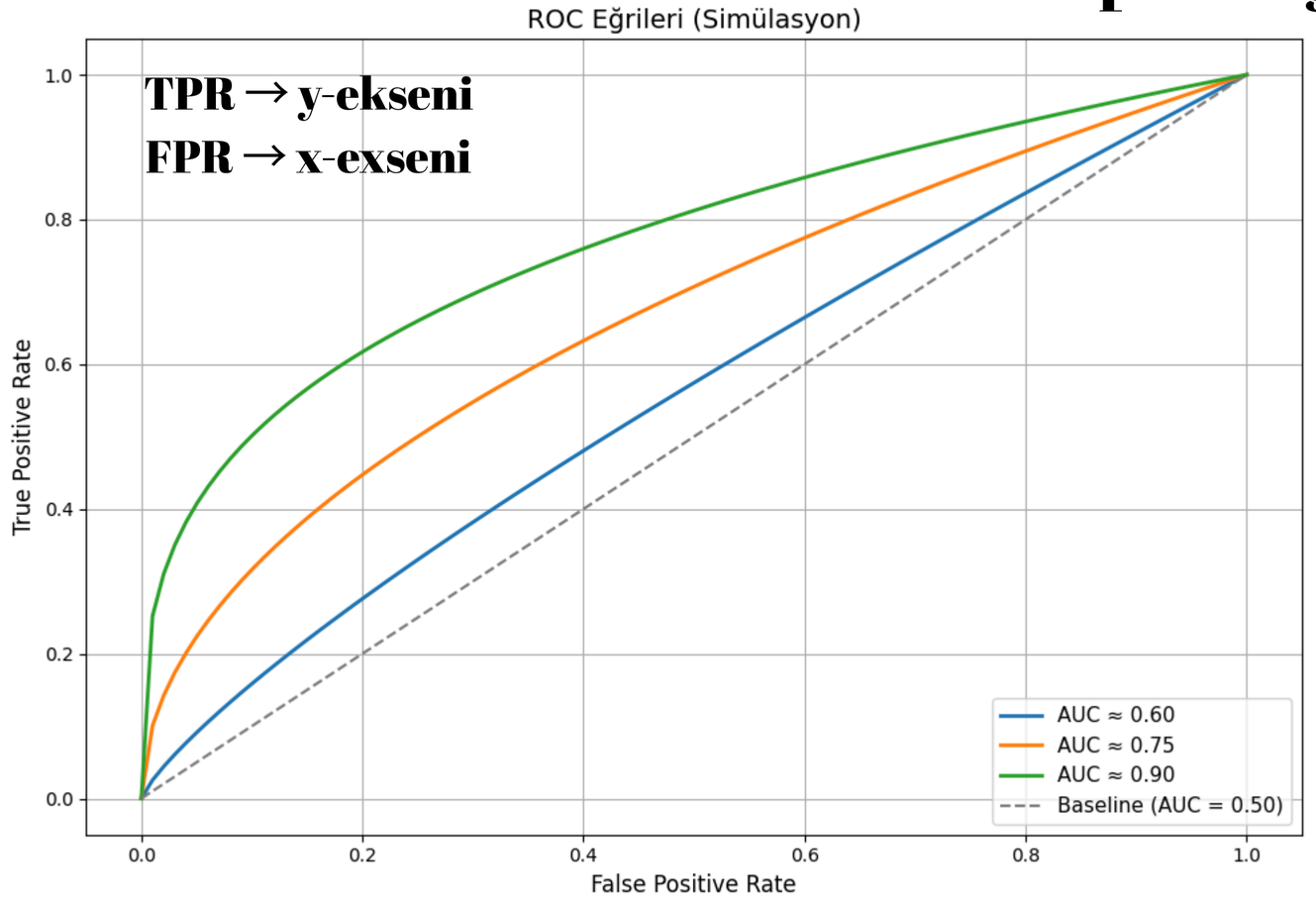
$$(0.5 + 0.5 + 1.0) / 3 \approx 0.667$$

Bu örnekte destekler eşit; weighted F1 = macro F1.

Örnek ağırlıkları (sample_weight) ayrı bir kavramdır.

ROC

Receiver operating characteristic



$$\text{True Positive Rate (TPR)} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

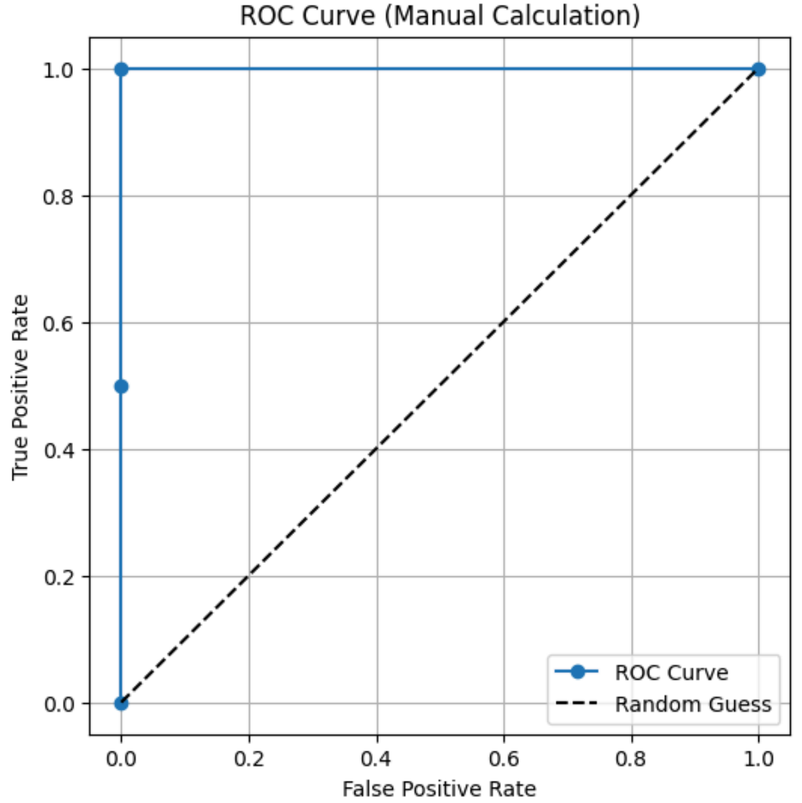
$$\text{False Positive Rate (FPR)} = \frac{\text{False Positive}}{\text{False Positive} + \text{True Negative}}$$

*sensitivity = Recall
*specificity = 1-FPR

- ROC, farklı eşiklerde TPR ve FPR'yi gösterir.
- Olasılık veya eşik uygulanmamış karar skoru kullanılır.
- Yalnızca sınıf etiketleri tek bir çalışma noktası verir.

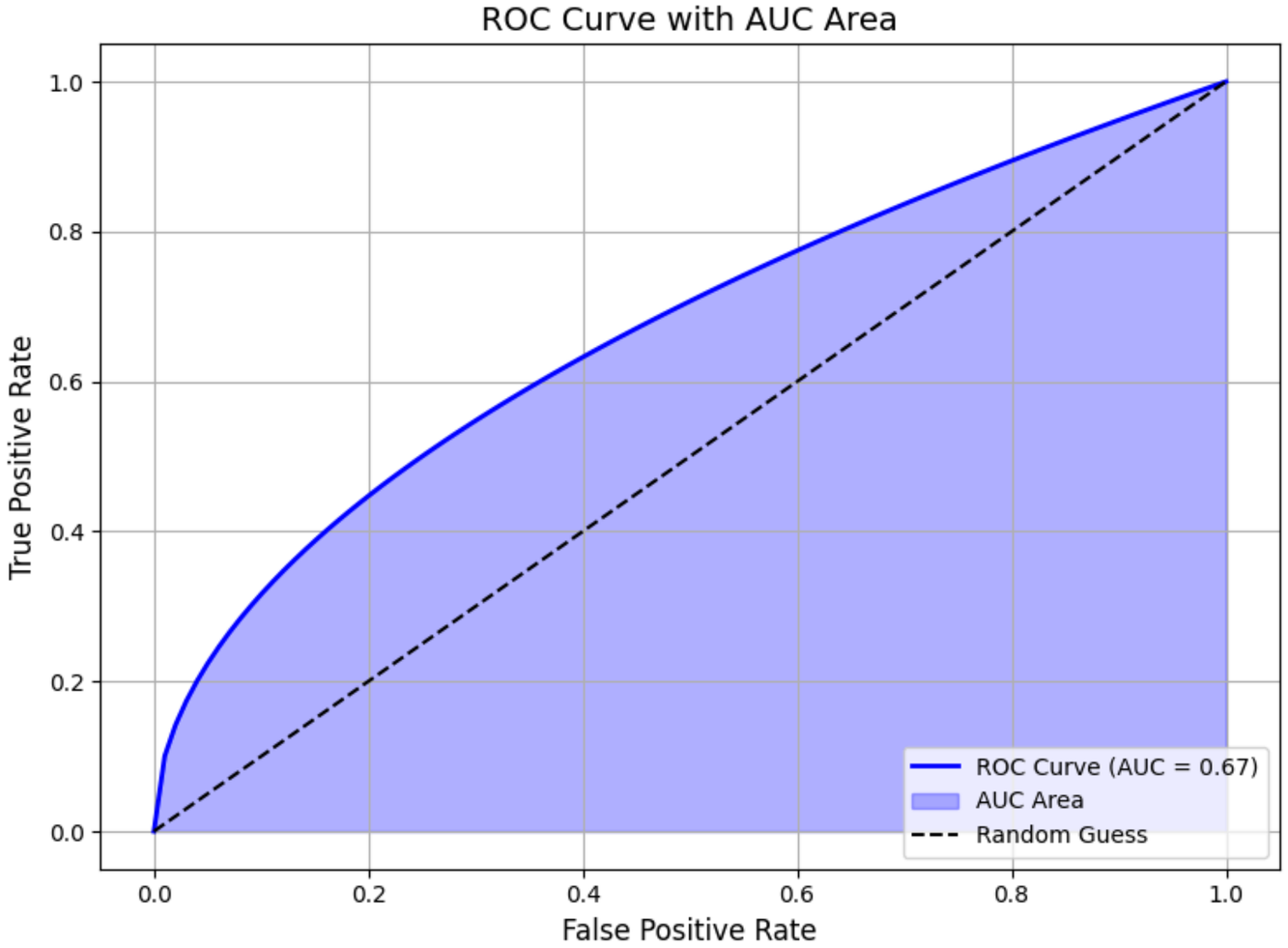
ROC
Receiver operating characteristic

Ground Truth	Prediction	Eşik=0.1	Eşik=0.3	Eşik=0.5	Eşik=0.7
1	0.4	1	1	0	0
1	0.6	1	1	1	0
0	0.2	1	0	0	0
		TPR=1 FPR=1	TPR=1 FPR=0	TPR=0.5 FPR=0	TPR=0 FPR=0



Üç örnekli grafik hesap mantığını gösterir.
Notebookta altı temsili skorla ROC-AUC çiziyoruz.

AUC
Area under the curve



Roc eğrisinin altında kalan alandır
Tek bir sayısal değer ifade eder. Bu yüzden yorumlaması kolaydır.

Yüksek AUC → Daha iyi model

AUC eğriyi tek sayıyla özetler.
ROC-AUC: 1 mükemmel sıralama; 0.5 rastgele sıralama.

PR-AUC ve Average Precision aynı hesap değildir.
Bu dönem ROC-AUC'nin grafik yorumuna odaklanıyoruz.

Train-Validation-Test



Train → modeli eğitmek için kullanılır

Validation → hyperparametreleri ve modelleri seçmek için kullanılır

Test → Seçilen sistemin nihai performansını değerlendirmek için kullanılır

- **Notebook: %60 train, %20 validation, %20 test.**
- **LogisticRegression için C değerlerini karşılaştırırız.**
- **Model train'de öğrenir; seçenekler validation / CV'de seçilir.**
- **Test, seçilen sistemin son değerlendirmesine ayrılır.**

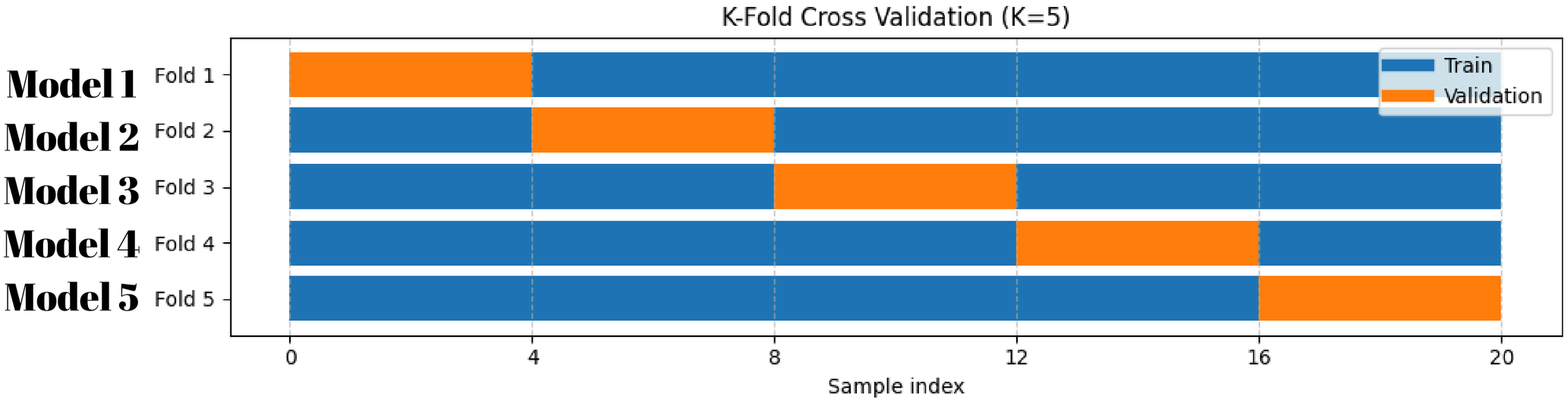
Train-Validation-Test

1. Train: model parametrelerini öğren.
2. Validation / CV: model ve hiperparametre seç.
3. Test: son değerlendirme; seçenekleri testte seçme.
4. Bölme oranları veri ve uygulamaya göre değişir.
5. Doldurma / scaling / encoding yalnızca train'den öğrenilir.
6. Aynı dönüşümler validation ve test'e uygulanır.

Verilerin bu setlere uygun dağıtılması önemlidir. Sonraki konular bunla alakalı →

K-fold Cross Validation

Önce son test setini ayır. Kalan eğitim havuzunu k parçaya böl.
Her turda bir parça validation, kalanlar train olur.



Farklı bölmelerden k skor elde edilir; ortalama ve yayılımı yorumlarız. Notebookta k=3, görselde k=5.

K-fold, her aday için K ayrı model eğitir.
Hesaplama maliyeti artar; süre veri boyutu ve modelin eğitim hızına bağlıdır.

K-fold Cross Validation

- 1. Hiperparametreleri eğitim havuzunda CV ile karşılaştır.
- 2. En iyi ayarlarla tüm eğitim havuzunda yeniden eğit.
- 3. Son modeli ayrılmış test setinde değerlendir.
- Repeated Stratified K-Fold: farklı bölmelerle tekrar; bu dönem yalnızca isim.

Stratified K-Fold

Sınıf oranlarını her fold'da mümkün olduğunca korur.

	1. Fold	2. Fold	3. Fold	4. Fold	5. Fold	Toplam
Pozitif	1	1	1	1	1	5
Negatif	10	10	10	10	10	50

Notebookta StratifiedKFold(n_splits=3) kullanıyoruz.

Fold'ların train ve validation örnekleri birbirinden ayrıdır.

Belirsizlik: Fold Skorlarının Yayılımı

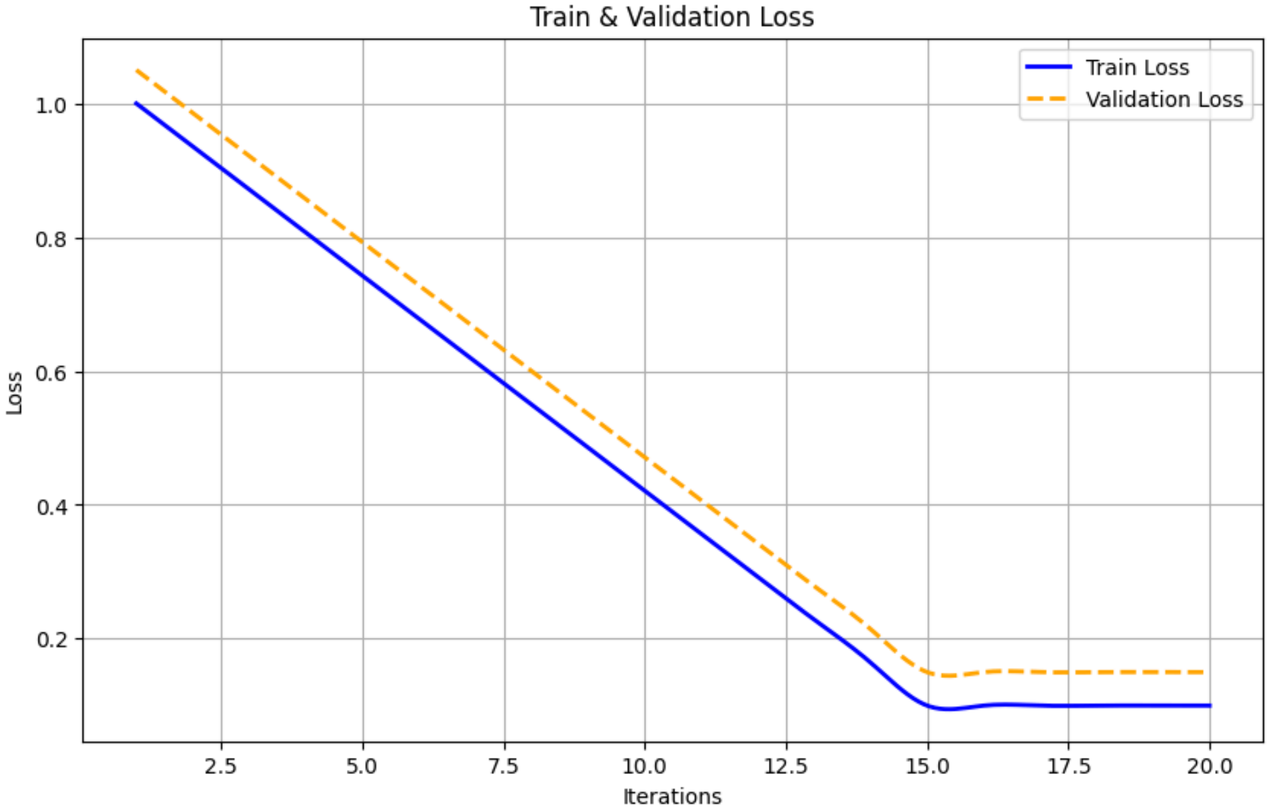
Farklı fold skorları, veri bölmesine duyarlılığı gösterir.
Ortalama ve standart sapmayı birlikte yorumlarız.

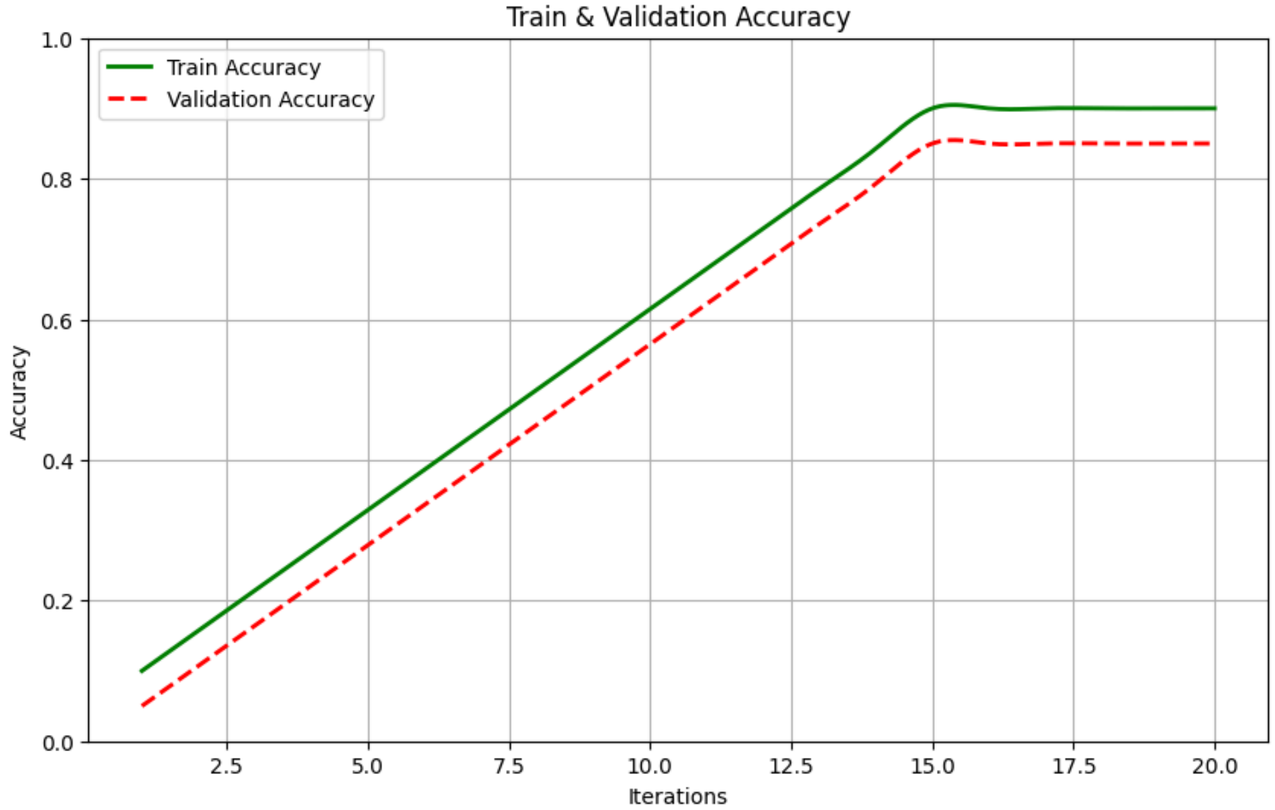
Güven aralığı, bir tahminin belirsizliğini ifade eder.
Fold'lar bağımsız olmadığı için std doğrudan güven aralığı değildir.

Örnek ortalama ≈ 0.892
Örnek std ≈ 0.059
Bu değerler %95 güven aralığı değildir.

Fold	Accuracy
Fold 1	0.90
Fold 2	0.88
Fold 3	0.92
Fold 4	0.80
Fold 5	0.96

Train-Val Graphs





Her iteration için loss kaydet
Eğitim bittikten sonra grafiği çizdir

Her iteration için accuracy kaydet
Eğitim bittikten sonra grafiği çizdir

Bunlar temsili grafiklerdir. Train ve validation birlikte izlenir;
grafik şekli tek başına iyi model kanıtı değildir.

Bu grafikler eğitim anlaşılmasında çok önemlidir!!!

Genelleme

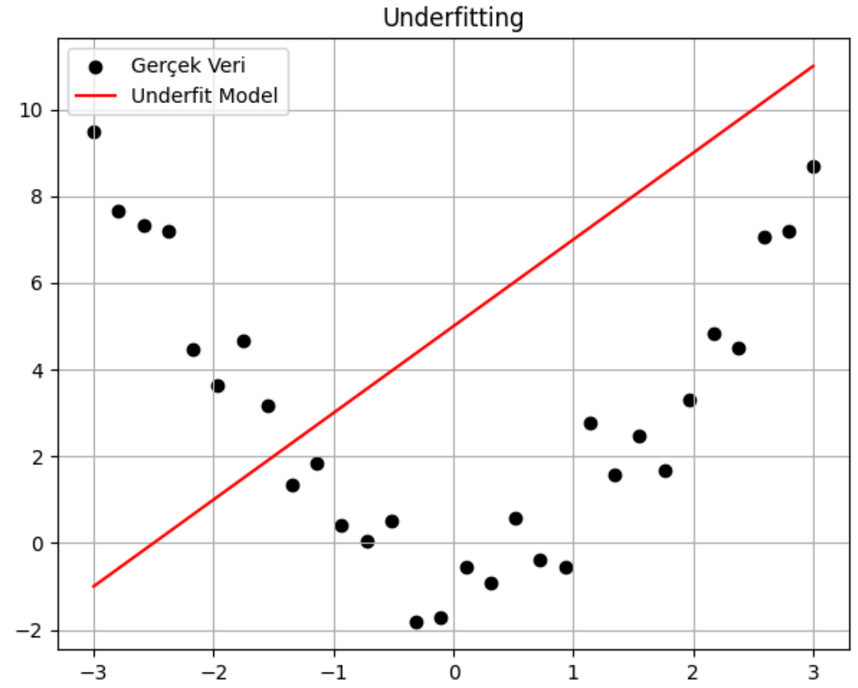
Genelleme: daha önce görülmemiş veride başarılı olmak.

Underfitting: train ve validation'da yetersiz uyum.

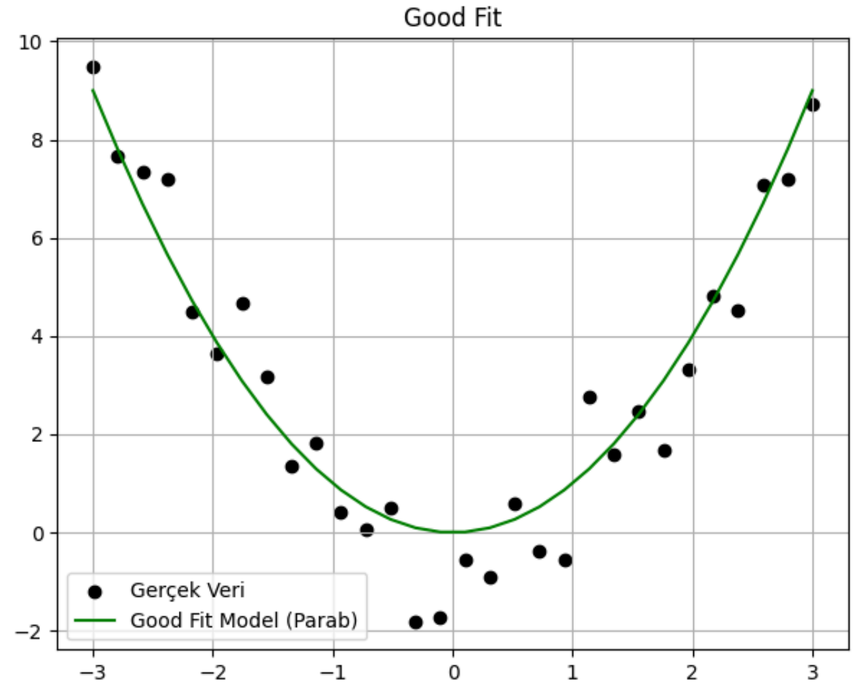
Overfitting: train'de iyi, yeni veride daha zayıf performans.

Modül 1'deki loss düşüşü yalnızca eğitim uyumunu gösterir.

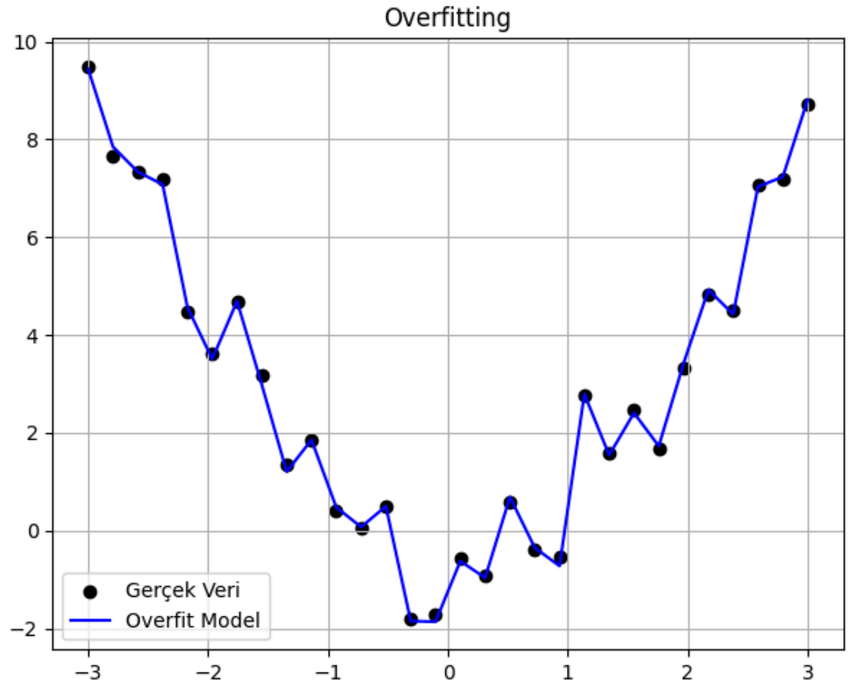
Genelleme nasıl görünüyor



Yeterli öğrenememiş



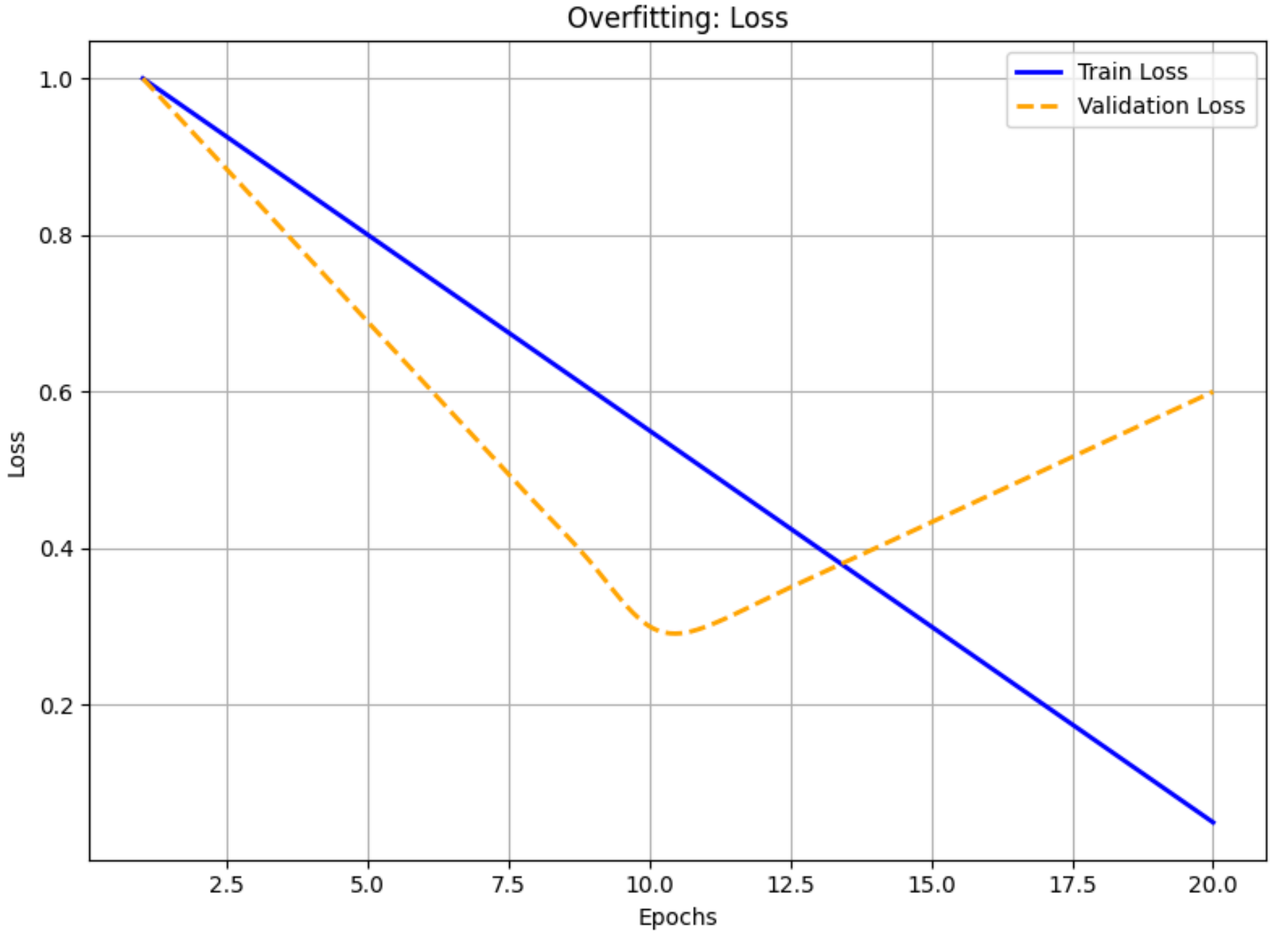
Güzel öğrenmiş

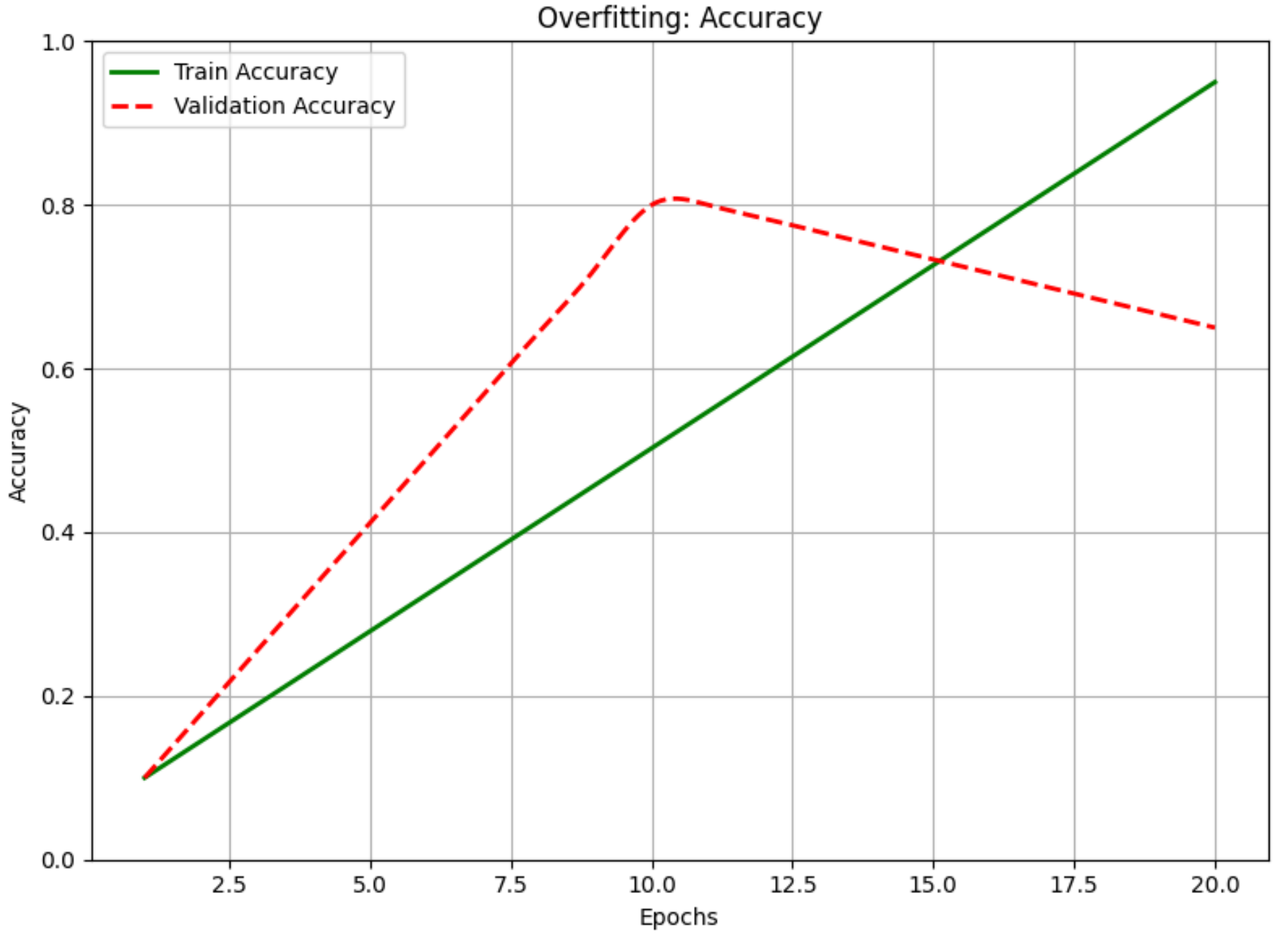


Uydurarak öğrenmiş

Görmediği yeni veride patlar

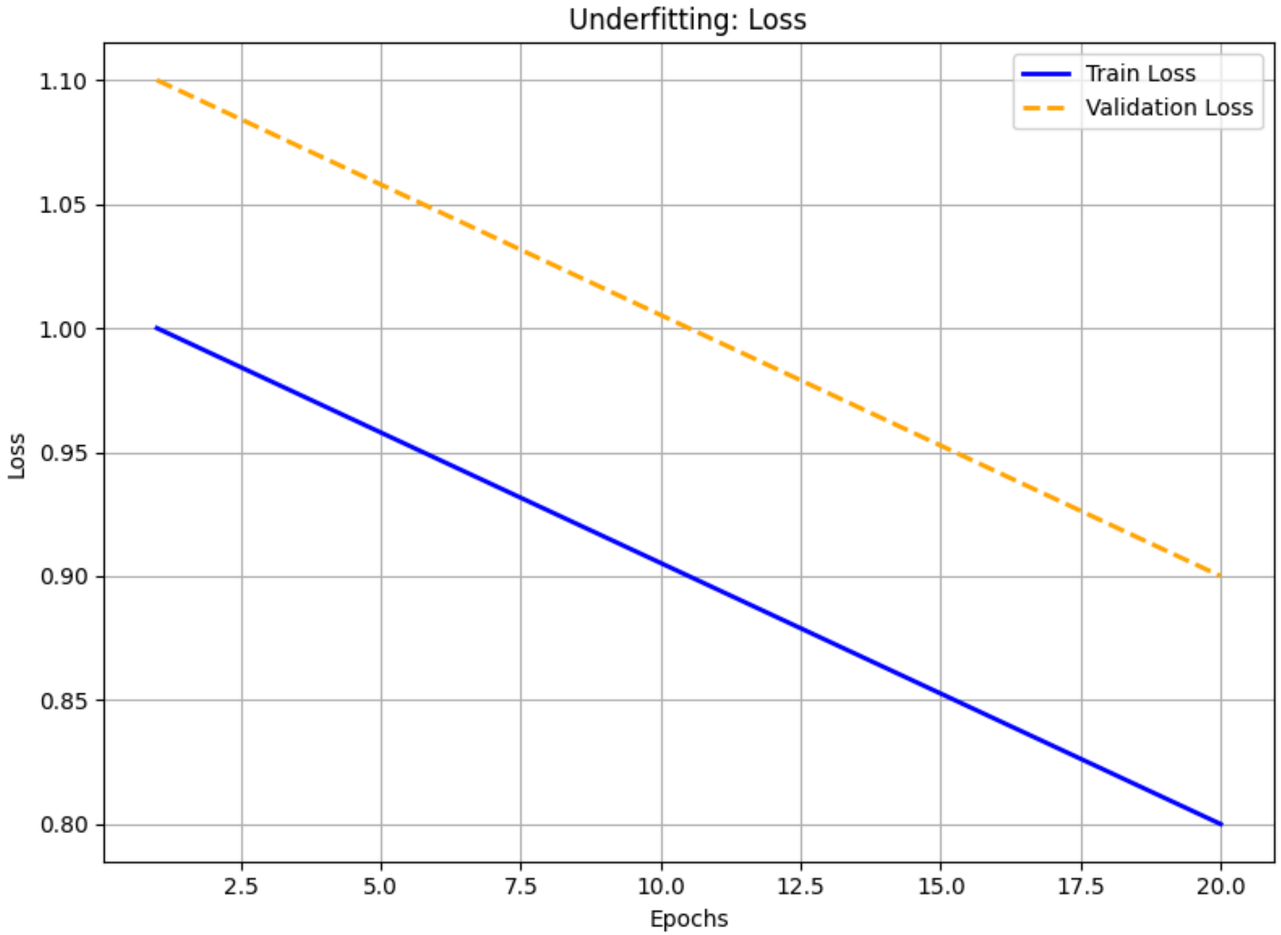
Overfitting

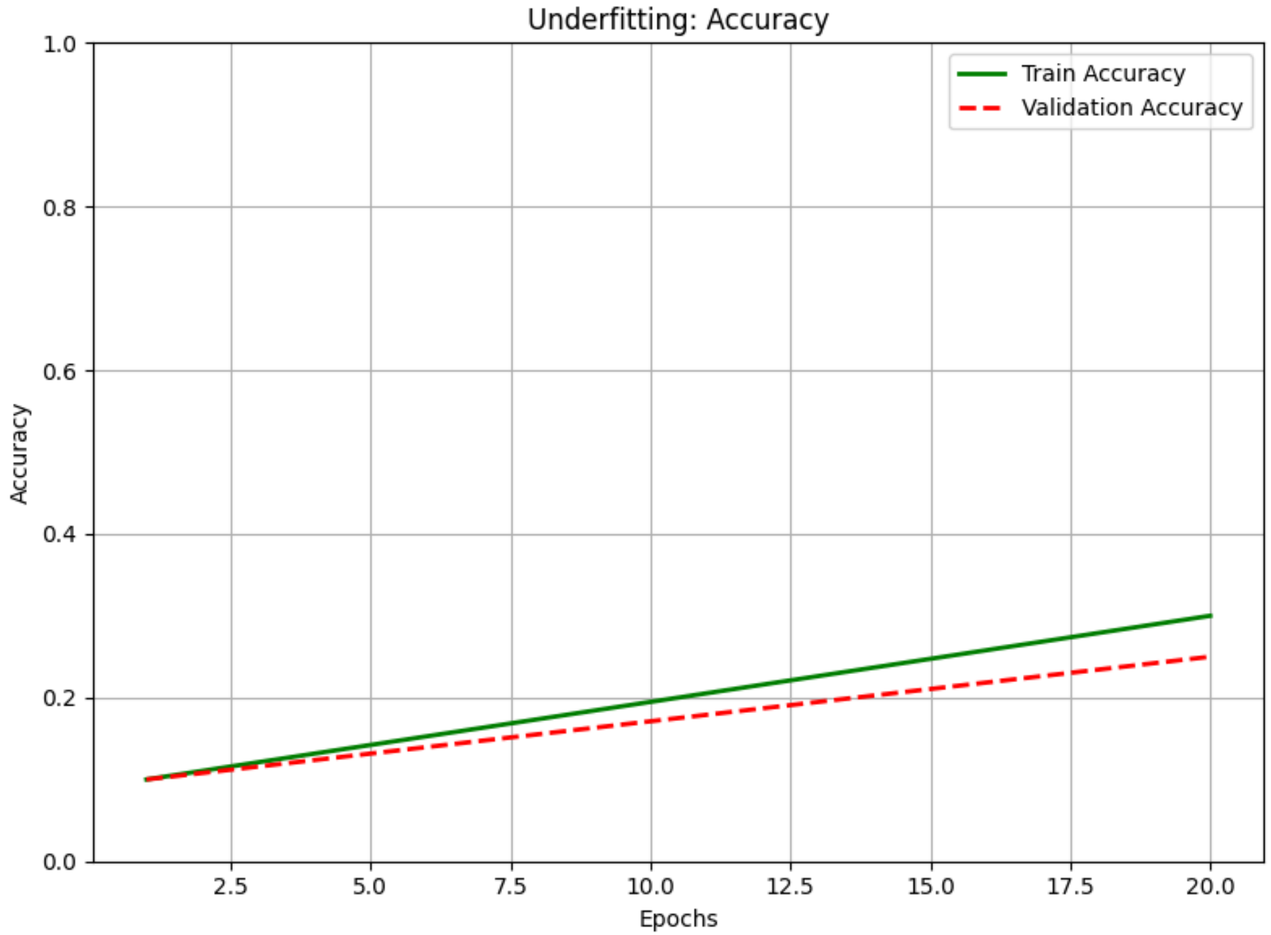




Train loss düşerken validation loss'un yükselmesi overfitting işareti olabilir.

Underfitting





Train ve validation başarısı yetersizse underfitting olabilir.
Daha uzun eğitim bazı durumlarda yardımcı olur; her zaman çözmez.

Bazı underfitting durumlarında eğitim devam etse bile öğrenememe durumu olabilir. Bu da modelin sınırlarını gösterir.

Underfitting-Overfitting çözümleri

Underfitting	Overfitting
Model karmaşıklığını artırmak.	Model karmaşıklığını azaltmak
Regularization azaltmak	Regularization artırmak
Eğitim süresini uzatmak	Eğitim süresini azaltmak
Veri temsilini iyileştir	Gereksiz özellikleri azalt
C’yi artırmayı dene	C’yi azaltmayı dene

Sonuçları validation / CV’de karşılaştır

LogisticRegression: küçük C = güçlü regularization

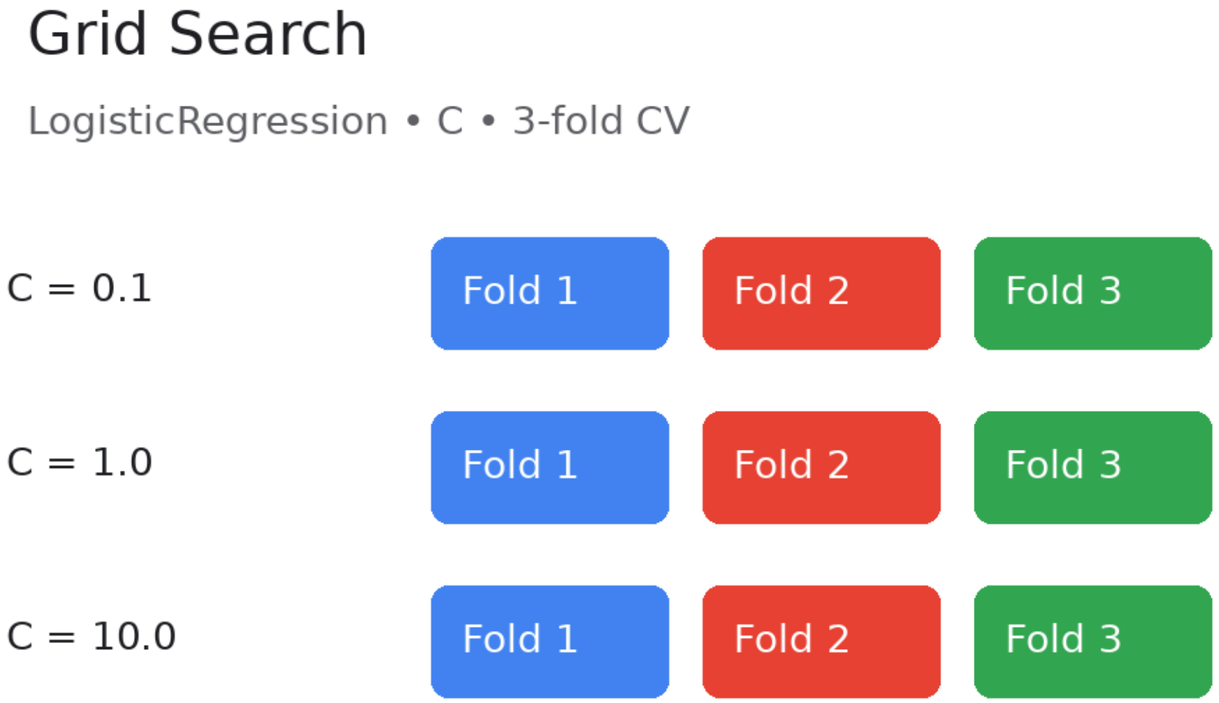
Hyperparameter Search

Parameter → Model tarafından eğitim sırasında güncellenerek öğrenilen sayılardır. (Ör: Weight-bias)

Hiperparametre: eğitimden önce seçilen ayar. Örnek: C.

C=[0.1, 1, 10] değerlerini CV ile karşılaştır.
Küçük C → daha güçlü regularization.
En iyi seçeneği test skoruna bakarak belirleme.

Grid Search



Önceden belirtilmiş değerlerin her birini kombinasyon hesabı ile denemek

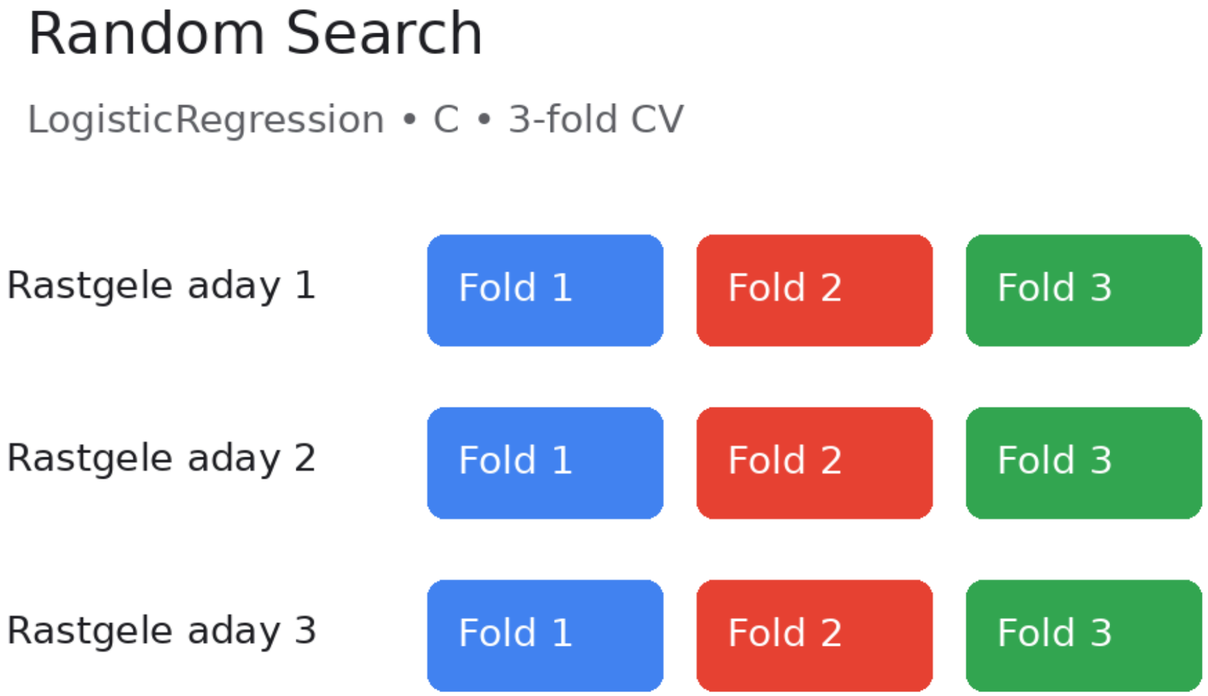
param_grid = {"C": [0.1, 1.0, 10.0]}
cv = 3; scoring = "accuracy"

3 aday × 3 fold = 9 eğitim.
Seçilen ayarla tüm eğitim havuzunda 1 refit.

Grid yalnızca belirttiğimiz C değerlerini dener;
örneğin C=0.5 bu listede yoktur.

Çok fazla değer denemek işlem sürecini çok fazla uzatabilir.

Random Search



Notebook: C adaylarından rastgele 3 değer seçilir.
Her aday 3-fold CV ile değerlendirilir.

Geniş parametre uzaylarında eğitim sayısını sınırlar.
Tüm kombinasyonları denemez; en iyiyi bulma garantisi yoktur.

Örnek: n_iter=3 ve cv=3 → 9 eğitim + 1 refit.
Bayesian arama, önceki sonuçlardan yararlanarak sonraki adayları seçer.

Bayesian optimization

Bayesian optimization: önceki sonuçlara göre yeni aday seçer.

Bu dönem yalnızca yaklaşımın adını biliyoruz.

Ödev 2: LogisticRegression → Accuracy/F1 → GridSearch.

Ödev: C=[0.1,1,10], cv=3, accuracy; arama yalnızca train'de.

Optuna: A Next-generation Hyperparameter Optimization Framework